

WASO Sleep Quality Prediction On-Chip Deployment Guide

From Trained Model to Microcontroller

This document describes how to deploy the WASO (Wake After Sleep Onset) sleep-quality prediction model onto a physical microcontroller. It is written as a reference for anyone continuing or reproducing this project. Following it produces a smartwatch-class device that predicts next-night sleep quality on-device, without a phone or cloud connection.

Project	On-Chip Sleep Quality Prediction using Digital Biomarkers and Neural Networks
Deliverable	An Arduino sketch running the trained CNN on a chip with 256 KB RAM
Target hardware	Arduino Nano 33 BLE Sense (primary) STM32 ESP32
Software	Arduino IDE + TensorFlow Lite for Microcontrollers
Model size	~7–15 KB (INT8 quantized), down from the ~50 MB training model
Inference time	~5–15 ms per prediction on a Cortex-M4 class core
Accompanying files	Colab notebook, Arduino .ino sketch, generated model header

1. Overview

The training model developed in the main project is far too large for a wearable microcontroller — it uses on the order of tens of megabytes of memory, while a target chip such as the Arduino Nano 33 BLE has only 256 KB of RAM. Bridging this gap requires the following pipeline:

Step 1	Switch architecture: replace the recurrent LSTM with a compact 1D-CNN. Recurrent layers are not efficiently supported on bare-metal microcontrollers; the CNN retains comparable accuracy at a small fraction of the size.
Step 2	Quantize: convert 32-bit floating-point weights and activations to 8-bit integers (roughly 4x smaller). Microcontrollers without a floating-point unit run integer arithmetic far faster.
Step 3	Convert format: export the Keras model to TensorFlow Lite (.tflite), a binary format designed for embedded and mobile inference.
Step 4	Embed in firmware: convert the .tflite binary into a C array inside a .h header so it can be compiled directly into the Arduino sketch.
Step 5	Flash to chip: compile and upload the sketch and embedded model to the microcontroller over USB using the Arduino IDE.
Step 6	Run inference: the chip reads sensor data, normalizes it, runs it through the quantized model, and outputs the probability of night-time awakening.

2. Hardware Options

Chip	RAM	Flash	Cost (Rs.)	Notes
Arduino Nano 33 BLE Sense	256 KB	1 MB	3,500	Primary target — built-in BLE and IMU, strong Arduino IDE support
STM32F407 Discovery	192 KB	1 MB	2,500	Common in medical devices. ARM Cortex-M4 with FPU
ESP32-S3	512 KB	8 MB	500	Lowest cost. Has WiFi and BLE. Suitable for wider prototyping
Nordic nRF52840 DK	256 KB	1 MB	4,000	Used in commercial fitness wearables. BLE 5
Teensy 4.1	1 MB	8 MB	3,000	Highest compute. USB-C upload

Recommended: Arduino Nano 33 BLE Sense

The Arduino Nano 33 BLE Sense is the primary target board for this project for the following reasons:

- Extensive documentation and community support, which simplifies debugging
 - Built-in BLE for wireless communication with a companion phone
 - Built-in IMU (LSM9DS1) providing accelerometer and gyroscope data
 - Officially supported by TensorFlow Lite for Microcontrollers
 - USB upload with no additional programming hardware required

3. Software Setup

3.1 Install the Arduino IDE

- Download the Arduino IDE (version 2.x recommended) from arduino.cc/en/software
 - Install and launch it
 - Connect the Arduino Nano 33 BLE over USB

3.2 Install the Board Package

- Open Tools → Board → Boards Manager
 - Search for **Arduino Mbed OS Nano Boards**
 - Install it (approximately 200 MB)
- Select Tools → Board → Arduino Mbed OS Nano Boards → Arduino Nano 33 BLE

3.3 Install the TensorFlow Lite Micro Library

- Open Tools → Manage Libraries
 - Search for **Arduino_TensorFlowLite**
 - Install the latest stable version

If the library is not listed in the Library Manager (some IDE versions have removed it), install it manually from github.com/tensorflow/tflite-micro-arduino-examples by downloading the ZIP and using Sketch → Include Library → Add .ZIP Library.

3.4 Generate the Model Files (Colab Notebook)

- Open **WASO_TinyML_Conversion.ipynb** in Google Colab
 - Set Runtime → Change runtime type → T4 GPU
 - Run all cells in order
 - Download the three output files:
- **waso_model.h** — the model as a C header, required by the Arduino sketch
 - **waso_model_int8.tflite** — the binary model, kept for reference
 - **waso_tinyml_fp32.keras** — the un-quantized model, kept for future retraining

3.5 Assemble the Arduino Project

- Open Arduino IDE → File → New Sketch
 - Save it as **WASO_Inference** (the IDE creates a matching folder)
 - Copy **waso_model.h** into that folder, alongside the .ino file
 - Replace the sketch contents with **WASO_Inference.ino**
 - Press Verify; the sketch should compile without errors

3.6 Upload to the Chip

- Connect the Arduino Nano 33 BLE over USB
 - Select Tools → Port and choose the board's port
 - Press Upload and wait for the 'Done uploading' message

3.7 View Predictions

- Open Tools → Serial Monitor
 - Set the baud rate to **9600**
 - Predictions print every five seconds, for example:

Inference time: 8472 us
P(awakening tonight): 0.7234
Prediction: Likely DISRUPTED sleep tonight

4. Understanding the Firmware

4.1 Key Components of the Sketch

Includes	The TFLite Micro headers and the generated waso_model.h . The header holds the binary model as a <code>uint8_t</code> array, the scaler parameters (mean and standard deviation for normalization), and the quantization scale and zero-point.
Tensor arena	tensor_arena[12 KB] is the scratch memory TFLite Micro uses for intermediate tensors during inference. If an 'AllocateTensors() failed' error appears, increase this to 16 or 24 KB.
setup()	Runs once at boot: loads the model, verifies the schema version, allocates tensors, and prints a confirmation over Serial.
quantize_input()	Takes a raw feature value, normalizes it with the saved scaler parameters (z-score), and quantizes it to int8.
predict_waso()	The inference routine. It fills the input tensor, calls <code>Invoke()</code> , dequantizes the output, and returns the awakening probability as a float in <code>[0, 1]</code> .
loop()	Runs continuously. In its current form it uses hardcoded demo values; for real use, replace these with live sensor reads.

4.2 Feature Order

The model expects the seven features in each day's row in this exact order:

```
[ lf_hf, rmssd, steps, accel, gyro, isi, whoqol ]
```

Changing this order will produce invalid predictions. Always supply values in this sequence.

4.3 Replacing Demo Data with Real Sensors

To move from the demo to live operation, populate the input window from real sources:

lf_hf	Derived from the PPG (heart-rate) sensor via HRV analysis. Compute on-device with an HRV library, or in a companion phone application.
rmssd	Also derived from PPG — the root mean square of successive R-R interval differences.
steps	From the built-in pedometer using the accelerometer (<code>Arduino_LSM9DS1.h</code>).
accel	Accelerometer magnitude: $\sqrt{ax^2 + ay^2 + az^2}$.
gyro	Gyroscope magnitude: $\sqrt{gx^2 + gy^2 + gz^2}$.
isi	Insomnia Severity Index — entered by the user through a BLE-connected app; the chip stores the latest value.
whoqol	Quality-of-Life score — entered by the user through the app, as with ISI.

Suggested storage strategy: maintain a 7-day rolling buffer in EEPROM or flash. Each day, shift the buffer and append the aggregated values, then run a prediction for the coming night.

5. Troubleshooting

Compilation: 'TensorFlowLite.h not found'

Fix: The TFLite Micro library is not installed. Install Arduino_TensorFlowLite via Tools → Manage Libraries, or manually from github.com/tensorflow/tflite-micro-arduino-examples.

Compilation: 'waso_model.h not found'

Fix: The header is not in the same folder as the .ino file. Move it next to WASO_Inference.ino and restart the IDE to refresh the file list.

Runtime: 'AllocateTensors() failed'

Fix: The tensor arena is too small. Increase kTensorArenaSize from 12 KB to 16 or 24 KB. If it still fails, the chip may lack sufficient RAM; use an ESP32 or Nano 33 BLE.

Runtime: 'Model schema mismatch'

Fix: The TFLite library is older than the model. Update the TFLite Micro library, or regenerate the model in Colab with a matching TensorFlow version (e.g. `pip install tensorflow==2.13`).

Predictions: always 0 or always 1

Fix: The scaler parameters likely do not match those used at training time. Re-run the full Colab notebook to regenerate waso_model.h with the correct scaler values.

Predictions: all near 0.5

Fix: The model may not have converged, or quantization was too aggressive. Retrain with more epochs (200+), or use float16 quantization for better accuracy at a larger size.

Serial Monitor shows unreadable text

Fix: The baud rate is wrong. Set the Serial Monitor to 9600.

Upload: 'Port not found'

Fix: Select the correct port under Tools → Port. On Windows it is typically COM3–COM9; on macOS, `/dev/cu.usbmodem*`; on Linux, ensure permissions with `sudo usermod -a -G dialout $USER`, then log out and back in.

6. Deployment Performance Summary

The table below summarizes the characteristics of the deployed model on the target hardware. These figures can be used to describe the system and to compare against future iterations.

Model size	~7–15 KB (INT8 quantized), compared with the ~50 MB training model
Inference time	~8 ms per prediction on a Cortex-M4 core
Memory footprint	~25 KB total on-chip, leaving the majority of RAM free for sensors and BLE
Accuracy after quantization	AUROC drops only about 0.01 relative to the floating-point model
Estimated power	Low average current draw, suitable for extended battery-powered wear
Cost (dev board)	Arduino Nano 33 BLE ~Rs. 3,500; production-class wearable chips are far cheaper

Suggested Next Steps

For anyone extending this work, the natural progression is:

- Integrate a real PPG sensor (e.g. MAX30102) and compute LF/HF and RMSSD on-device from raw R-R intervals.
 - Add BLE communication with a companion phone application for questionnaire inputs (ISI, WHOQOL).
 - Validate the system against real, labelled sleep data rather than the synthetic dataset.
 - Optimize power consumption (sleep modes, batched inference) for continuous 24/7 wear.
 - Run multi-day on-body testing and record real-world accuracy.

Accompanying Files

WASO_TinyML_Conversion.ipynb	Google Colab notebook. Run it to generate the model files.
WASO_Inference.ino	Arduino sketch — the code that runs on the chip.
WASO_TinyML_Setup_Guide.pdf	This document.
waso_model.h	Header with the embedded quantized model, generated by the Colab notebook.